

UD 5. Prácticas

Índice de contenidos

Prácticas.....	2
Práctica 1.....	2
Práctica 2.....	2
Práctica 3.....	2
Práctica 4.....	3
Práctica 5.....	3
Práctica 6.....	3
Práctica 7 (Requiere hacer Ejercicio 1 de Ampliación).....	3
Práctica 8 (Requiere hacer Ejercicio 2 de Ampliación).....	3
Práctica 9 (Requiere hacer Ejercicio 3 de Ampliación).....	4
Ejercicios de refuerzo.....	5
Refuerzo 1: Calculadora de Operaciones Básicas.....	5
Refuerzo 2: Mostrar una Lista de Elementos.....	5
Refuerzo 3: Detalles de un Elemento.....	5
Refuerzo 4: Generador de Mensajes Aleatorios.....	5
Refuerzo 5: Mostrar Productos y Detalles.....	6
Refuerzo 6: Convertidor de Unidades.....	6
Refuerzo 7: Actividad de usuario.....	6
Refuerzo 8: Mi primer formulario.....	7
Refuerzo 9: Formulario de contacto.....	7
Refuerzo 10: Calculadora ampliada.....	7
Ejercicios de ampliación.....	8
Ejercicio 1: Relaciones entre entidades.....	8
Introducción.....	8
Tipos de relación.....	8
Creando los atributos.....	8
Persistiendo entidades relacionadas.....	11
Mostrar entidades relacionadas en twig.....	12
Ejercicio 2: Cargando datos de prueba con Seeding.....	14
Ejercicio 3: Autenticación.....	16
Usuario.....	16
Autenticación.....	17
Crear usuarios por defecto.....	18
Restringiendo el acceso.....	20
Restringir por patrón de ruta.....	20
Restringir en twig.....	21
Obteniendo el usuario actual.....	22
En el controlador.....	22
En las plantillas.....	22

Prácticas

La práctica de este tema consiste en replicar el miniblog que hicimos en PHP en la UD3 con Laravel (Práctica 5 de la UD3) y añadir alguna funcionalidad más. Cada práctica es una de las etapas del proceso.

Práctica 1

Crea un proyecto Laravel llamado *laravelBlog* y haz las siguientes cosas:

- Coloca el archivo css en la carpeta public.
- Crea una clase controlador llamada `BlogController` con dos métodos, uno para la portada y otro para ver una noticia. El primero en la ruta `/` y el segundo en la ruta `/noticia/{id}`.
- Crea una plantilla llama `blogBase.blade.php` que contenga la parte del HTML que se generaba con `theHeader()` y `theFooter()`. Añade un bloque para poder cambiar el título (etiqueta `title`) de la página (que por defecto sera *Laravel HackBlog*) y otro para el `div main`. Genera el enlace de la hoja de estilos usando `asset()`.
- Crea una plantilla para la portada y otra para ver una noticia, que hereden de `blogBase.blade.php`. De momento no mostrarán la noticias, sino un mensaje que diga "Esta es la portada" o "Esta es una noticia". El título de las páginas (etiqueta `title`) será "Portada - *Laravel HackBlog*" y "Noticia - *Laravel HackBlog*" respectivamente.

Práctica 2

- Crea la entidad *Noticia* (modelo y migración). Los campos `entradilla` y `cuerpo` deben ser de tipo `text`, mientras que el `titular` debe ser de tipo `string`. La fecha será de tipo `timestamp`.
- Crea un método `formNuevaNoticia()` en el controlador y una plantilla `formNuevaNoticia.blade.php` que muestre su formulario en la ruta `/nuevanoticia`. No se debe incluir un campo en el formulario para la fecha.

Práctica 3

Agrega un método `procesarFormNuevaNoticia()` para que almacene la noticia nueva en la base de datos y muestre una página de confirmación indicando que la noticia se ha guardado correctamente. La noticia debe tomar como fecha el valor de la fecha y hora en ese momento (puedes usar la función `now()`). Añade 3 noticias. Puedes comprobar si funciona bien consultando la base de datos.

Práctica 4

Modifica el método ver noticia para que tome el id de la URL, obtenga la noticia desde la base de datos, se la pase a la vista y la vista la muestre. Modifica la portada para que muestre la lista de noticias.

Práctica 5

Añade en la página de ver noticia un enlace para editar la noticia (/editarnoticia/id) y crea la funcionalidad.

Práctica 6

Añade en la página de ver noticia un enlace para borrar la noticia (/borrarnoticia/id) y crea la funcionalidad.

Práctica 7 (Requiere hacer Ejercicio 1 de Ampliación)

Crea la entidad `Comentario` y relacionalo con `Noticia` de forma que una noticia pueda tener varios comentarios, y los comentarios pertenezcan sólo a una noticia.

Deberás modificar la plantilla que muestra la noticia para que muestre también los comentarios y un formulario para añadir un comentario. Los comentarios deben procesarse en la misma página. Todos los comentarios deben pertenecer a una noticia y, si se borra la noticia, los comentarios deben borrarse también.

Práctica 8 (Requiere hacer Ejercicio 2 de Ampliación)

Crea una Seeder para cargar las tres noticias que venían de muestra en el proyecto original. Si has realizado los comentarios, carga también los comentarios de cada noticia.

Práctica 9 (Requiere hacer Ejercicio 3 de Ampliación)

Modifica la práctica:

- Crea un usuario con las siguientes propiedades:
 - email
 - nick
 - nombre
 - comentarios (relación OneToMany con comentarios, invertido por la propiedad autor)
 - noticias (relación OneToMany con comentarios, invertido por la propiedad autor)
- Añade con Fixtures un usuario normal y un usuario administrador.
- Controla que sólo el administrador pueda añadir, editar o borrar noticias. Al añadir la noticia se debe relacionar con el autor que lo hace. En la noticia se debe mostrar el autor de ella.
- Controla que el formulario de añadir comentarios sólo se muestre a usuarios registrados. Si no está registrado, muestra un mensaje diciendo que debe estar registrado para comentar, y un enlace al formulario de login. Cuando se añade un comentario, se relaciona con el usuario que lo añade.
- Muestra en los comentarios el nick del autor, que sea un enlace a enviarle un correo electrónico. Si el comentario es del autor de la noticia, se debe destacar con una clase y un estilo CSS.
- Añade un enlace para iniciar sesión que se muestre en todas las páginas, y si el usuario está registrado, que se muestre su nombre y un enlace para cerrar sesión.

Ejercicios de refuerzo

- Del 1 al 7: relacionados con separar Vista y Controlador sin usar formularios.
- Del 8 al 10: relacionados con separar Vista y Controlador, usando formularios.

Refuerzo 1: Calculadora de Operaciones Básicas

Implementa una ruta `/calculate/{num1}/{num2}/{operation}`. Dependiendo del parámetro `operation`, realiza suma, resta, multiplicación o división. Muestra el resultado en una vista.

Refuerzo 2: Mostrar una Lista de Elementos

Crea un controlador `ItemController` con un método que devuelva una lista de elementos desde un array. Muestra estos elementos en una vista con una ruta `/items`. El array podría ser por ejemplo:

```
$items = ['Manzanas', 'Plátanos', 'Peras', 'Naranjas'];
```

Refuerzo 3: Detalles de un Elemento

Usa el array del ejercicio anterior. Crea una ruta `/items/{id}` que muestre los detalles de un elemento específico. En el controlador, valida que el `id` existe en el array y pasa los datos a la vista. Si el `id` no existe, muestra un mensaje de "Elemento no encontrado".

Ejemplo de resultado:

Ruta `/items/1` muestra: Elemento seleccionado: Plátanos.

Refuerzo 4: Generador de Mensajes Aleatorios

Implementa una ruta `/random-message` que muestre un mensaje aleatorio de una lista. Define un array de mensajes en el controlador, como:

```
$messages = [  
    '¡Sigue adelante!',  
    'Hoy será un gran día.',  
    'Nunca dejes de aprender.',  
    'Confía en ti mismo.'  
];
```

Selecciona un mensaje al azar y pásalo a la vista. Muestra el mensaje aleatorio en la vista.

Refuerzo 5: Mostrar Productos y Detalles

En este ejercicio usarás un controlador llamado `ProductController`.

Usa un array para definir productos con sus datos: nombre, descripción y precio. Como por ejemplo:

```
$products = [
    ['name' => 'Laptop', 'description' => 'Una laptop potente', 'price' => 1200],
    ['name' => 'Mouse', 'description' => 'Mouse inalámbrico', 'price' => 20],
    ['name' => 'Teclado', 'description' => 'Teclado mecánico', 'price' => 50],
];
```

Muestra una lista de productos en formato de tabla en la ruta `/products`. Habrá una columna llamada “Ver detalles” que tendrá un enlace a la ruta `/products/{id}`.

Para `/products/{id}`, usa el `id` como índice para obtener los detalles de un producto y muéstralos en otra vista.

Refuerzo 6: Convertidor de Unidades

Implementa una ruta `/convert/{value}/{unit}` que convierta un valor dado a una unidad específica (por ejemplo: metros a kilómetros).

En el controlador, crea lógica para manejar conversiones de unidades como:

- km a m
- m a cm

Muestra el resultado de la conversión en una vista.

Ejemplo de resultado:

Ruta `/convert/1000/km` muestra: 1000 metros equivalen a 1 kilómetro.

Refuerzo 7: Actividad de usuario

En el controlador crea un array de usuarios con atributos como nombre y activo (`true/false`). Ejemplo:

```
$usuarios = [
    ['nombre' => 'Juan', 'activo' => true],
    ['nombre' => 'María', 'activo' => false],
    ['nombre' => 'Pedro', 'activo' => true],
];
```

Pásale el array a una vista llamada `usuarios.blade.php`. En la vista, muestra solo los usuarios que están activos con un mensaje: "Usuario Activo: [nombre]".

Refuerzo 8: Mi primer formulario

Crea una nueva ruta que apunte a un método `showForm` del controlador `UserController`. En el método `showForm`, devuelve una vista que tenga un formulario con un campo de texto (`name`) y un botón de enviar. En el controlador, agrega un método `storeName` que reciba la información del formulario y la muestre en pantalla.

Refuerzo 9: Formulario de contacto

Crea un formulario en la vista que recoja un nombre, correo electrónico y un mensaje. Este formulario se mostrará cuando el usuario acceda a la ruta `/contacto`.

Cuando se envíe y se procese el formulario en el método `storeForm` del controlador, se validará en el lado servidor que:

- El nombre sea de tipo texto y no supere la longitud de 30 caracteres.
- El correo electrónico sea un email.
- El mensaje sea de tipo texto y no supere la longitud de 500 caracteres.

Muestra un mensaje de agradecimiento con los datos ingresados en una nueva vista pero en la misma ruta de `/contacto`.

Refuerzo 10: Calculadora ampliada

Crea un formulario con dos campos numéricos y un menú desplegable para seleccionar una operación (suma, resta, multiplicación, división o potencia).

Cuando el usuario envíe el formulario, realiza la operación y muestra el resultado en la misma vista.

Asegúrate de validar en el lado servidor que los números sean números y que no estén vacíos. Si ocurriera algún error, en el formulario se mostrarían los errores que han ocurrido y se mantendrían los valores anteriores en caso de que estuvieran bien.

Ejercicios de ampliación

Ejercicio 1: Relaciones entre entidades

Introducción

Hay ocasiones en que las entidades de nuestro modelo están relacionadas, es decir, a una entidad A le corresponden ninguna, una o varias entidades B, que pueden ser o no del mismo tipo.

Con Doctrine no tenemos que preocuparnos por claves foráneas, punteros... únicamente añadir una entidad (o un array de ellas) como atributo de la otra y el tipo de relación, y Doctrine se encargará de *mapear* las entidades, es decir, de reflejarlas en la base de datos.

Tipos de relación

Para establecer relaciones entre entidades tenemos principalmente dos tipos (aunque hay más):

- 1:N** (o **N:1**): Relación **OneToMany** (o **ManyToOne**), en la que a un elemento de la entidad A le corresponden varios elementos de la entidad B y a la entidad B sólo un elemento de la entidad A (o viceversa). Es la relación más común y en bases de datos tradicionalmente se implementa incluyendo la ID de una entidad como atributo de la otra (clave foránea).
- N:M**: Relación **ManyToMany**, en la que a un elemento de la entidad A le corresponden varios elementos de la entidad B y viceversa. En este caso tradicionalmente se crea una tabla que contiene las IDs de ambas entidades.

Creando los atributos

Como ejemplo vamos a crear la entidad **Juego** (euromillones, bonoloto, primitiva...) de forma que a un juego le corresponden varias apuestas, y las apuestas corresponden a un sólo juego (relación **OneToMany**). Esta entidad tendrá dos atributos:

- Nombre: nombre del juego
- Apuestas: lista de apuestas (objetos de la Entidad **Apuesta**) relacionadas con el juego.

Para ello vamos a crear la entidad juego usando el asistente:

```
php bin/console make:entity
```

Ya sabemos cómo añadir el nombre del juego. Cuando añadamos el campo apuestas, debemos decir que es de tipo **relation** y que está relacionado con la entidad **Apuesta**:


```
Field type (enter ? to see all types) [string]:
```

```
> relation
```

```
What class should this entity be related to?:
```

```
> Apuesta
```

Nos explicará los diferentes tipos de relaciones, debemos seleccionar OneToMany

```
Relation type? [ManyToOne, OneToMany, ManyToMany, OneToOne]:
```

```
> OneToMany
```

Nos avisará de que modificará Apuesta para añadir una referencia al juego y nos preguntará cómo queremos llamarlo (por defecto pone el nombre de la entidad):

```
A new property will also be added to the Apuesta class so that you can access and set the related Juego object from it.
```

```
New field name inside Apuesta [juego]:
```

```
>
```

También nos preguntará si puede ser nulo (si pueden haber apuestas que no pertenezcan a ningún juego) y qué hacer con las apuestas cuando se borre un juego.

Cuando termine habrá creado una entidad Juego con sus *getters* y *setters*. A continuación una versión resumida, remarcando lo relacionado con Apuestas:

```
<?php
// src/Entity/Juego
// ...
class Juego
{
    // ...
    public function __construct()
    {
        $this->apuestas = new ArrayCollection();
    }

    /**
     * @ORM\OneToMany(targetEntity="App\Entity\Apuesta", mappedBy="juego",
orphanRemoval=true)
     */
    private $apuestas;

    /**
     * @return Collection|Apuesta[]
     */
    public function getApuestas(): Collection
    {
        return $this->apuestas;
    }
}
```

```

public function addApuesta(Apuesta $apuesta): self
{
    if (!$this->apuestas->contains($apuesta)) {
        $this->apuestas[] = $apuesta;
        $apuesta->setJuego($this);
    }
    return $this;
}

public function removeApuesta(Apuesta $apuesta): self
{
    if ($this->apuestas->contains($apuesta)) {
        $this->apuestas->removeElement($apuesta);
        // set the owning side to null (unless already changed)
        if ($apuesta->getJuego() === $this) {
            $apuesta->setJuego(null);
        }
    }
    return $this;
}
}

```

Como `Apuesta` es una lista, `$this->apuestas` es un atributo de tipo `ArrayCollection`. Es como un `Array`, pero con algunas funcionalidades más. Este código nos permite obtener la lista de apuestas, añadir una o borrar una. De esta manera, si queremos añadir una apuesta a un juego, lo haremos con `$juego->addApuesta($apuesta);`.

Si nos fijamos en el lado de `Apuesta`, veremos que se ha añadido un atributo y sus correspondientes *getters* y *setters* para poder hacer consultas y modificaciones:

```

<?php
// src/Entity/Apuesta

// ...
class Apuesta
{
    // ...

    /**
     * @ORM\ManyToOne(targetEntity="App\Entity\Juego", inversedBy="apuestas")
     * @ORM\JoinColumn(nullable=false)
     */
    private $juego;

    public function getJuego(): ?Juego
    {
        return $this->juego;
    }

    public function setJuego(?Juego $juego): self
    {
        $this->juego = $juego;

        return $this;
    }
}

```

De esta manera, si a una apuesta queremos asignarle un juego, lo haremos con

```
$apuesta->setJuego($juego);.
```

¡Atención!

Para los métodos `setJuego` o `addApuesta` los argumentos no son el id del juego a fijar o el id de la apuesta a añadir, sino el objeto juego o el objeto apuesta al completo.

Para que Doctrine refleje los cambios en la base de datos, deberemos crear y ejecutar las migraciones:

¡Atención!

Se recomienda que antes de ejecutar las migraciones vaciemos la tabla de las apuestas, ya que ninguna de las apuestas que hemos metido tiene juego asociado y es algo que, si hemos declarado que el atributo juego de Apuesta no puede ser nulo, puede darnos problemas.

```
php bin/console doctrine:migrations:diff
php bin/console doctrine:migrations:migrate
```

Persistiendo entidades relacionadas

Simplemente debemos hacer `persist` con ambas entidades. El siguiente código de ejemplo crea un juego llamado Euromillones y crea también una apuesta para ese juego:

```
// ...

class SorteoController extends AbstractController
{
    public function ejemplo(ManagerRegistry $doctrine)
    {
        $juego = new Juego();
        $juego->setNombre('Euromillones');

        $apuesta = new Apuesta();
        $apuesta->setTexto('2 4 14 15 31 10 12');
        $apuesta->setFecha(new \DateTime('tomorrow'));

        // Aquí es donde relacionamos la apuesta con el juego.
        $apuesta->setJuego($juego);

        $entityManager = $doctrine->getManager();
        $entityManager->persist($juego);
        $entityManager->persist($apuesta);
        $entityManager->flush();

        return new Response(
            'Juego creado con id: '.$juego->getId()
            .' y apuesta creada con id: '.$apuesta->getId()
        );
    }
}
```

Si una de las entidades la hemos obtenido de la base de datos, no hace falta persistirla. El siguiente código de ejemplo obtiene de la base de datos un juego a partir de su ID y crea una apuesta para ese juego:

```
// ...
use App\Entity\Apuesta;
use App\Entity\Juego;
use Symfony\Component\HttpFoundation\Response;

class SorteoController extends AbstractController
{
    public function ejemplo2($idJuego, ManagerRegistry $doctrine)
    {
        $entityManager = $doctrine->getManager();
        $juego = $entityManager->getRepository(Juego::class)->find($idJuego);

        $apuesta = new Apuesta();
        $apuesta->setTexto('2 4 14 15 31 10 12');
        $apuesta->setFecha(new \DateTime('tomorrow'));

        // Asigna a la apuesta el juego correspondiente.
        $apuesta->setJuego($juego);

        $entityManager->persist($apuesta);
        $entityManager->flush();

        return new Response(
            'Apuesta creada con id: '.$apuesta->getId()
        );
    }
}
```

Mostrar entidades relacionadas en twig

Si el atributo es una entidad, podemos acceder a ella directamente. Por ejemplo, podríamos modificar ``verApuesta.html.twig`` para obtener datos de un juego desde la apuesta:

```
<p>ID Apuesta: {{ apuesta.id }}</p>
<p>Juego: {{ apuesta.juego.nombre }}</p>
```

En el controlador no tendríamos que cambiar nada, ya que cuando Doctrine se *trae* un objeto de la base de datos, se trae también todos los objetos relacionados. La plantilla lo único que recibiría sería la apuesta:

```
// ...
public function verApuesta($id) {
    // ...
    $apuesta= $entityManager->getRepository(Apuesta::class)->find($id);
    // ...
    return $this->render('sorteo/verApuesta.html.twig', array(
        'apuesta' => $apuesta,
    ));
}
```

Para saber más

Cuando Doctrine se trae un objeto de la base de datos, se trae también todos los objetos relacionados. Es decir, cuando yo obtengo el Juego con ID 1, doctrine ejecuta consultas para traerse de la base de datos no sólo los datos de ese juego sino también de todas las apuestas relacionadas.

Esto hace más sencillo el trabajo del desarrollador, pero por contra empeora el rendimiento. Es posible optimizar esto señalando qué datos y qué entidades vamos a necesitar.

Debes crear un repositorio privado en GitHub y alojar en él este proyecto. Cuando añadas los archivos del proyecto, asegúrate de añadir los archivos ocultos (como .env o .gitignore).

Si el atributo es una lista (colección), se comportará como un array:

```
<h1>{{ juego.nombre }}</h1>
<ul>
{% for apuesta in juego.apuestas %}
    <li>Apuesta #{{ apuesta.id }}</li>
{% else %}
    <li>No hay apuestas</li>
{% endfor %}
</ul>
```

Práctica 7

Realiza la práctica 7.

Ejercicio 2: Cargando datos de prueba con Seeding

Hay veces que necesitamos cargar datos de golpe en la base de datos en lugar de introducirlos uno a uno, como por ejemplo para cargar una configuración inicial, cargar usuarios por defecto, cargar datos de muestra, cargar datos para tests...

Para estos casos Symfony cuenta con una extensión llamada Fixtures. Para añadirla a nuestro proyecto ejecutamos la siguiente línea:

```
composer require --dev doctrine/doctrine-fixtures-bundle
```

Con el siguiente comando se genera una clase *fixture* vacía:

```
php bin/console make:fixtures
```

Como ejemplo vamos a añadir una fixture a nuestro proyecto de sorteos que cargue dos apuestas de ejemplo, así que ejecutamos la línea anterior e indicamos que queremos crear la clase `ApuestaFixtures`. Tras hacerlo nos genera el archivo `src/DataFixtures/ApuestaFixtures.php`:

```
<?php

namespace App\DataFixtures;

use Doctrine\Bundle\FixturesBundle\Fixture;
use Doctrine\Persistence\ObjectManager;

class ApuestaFixtures extends Fixture
{
    public function load(ObjectManager $manager)
    {
        // $product = new Product();
        // $manager->persist($product);

        $manager->flush();
    }
}
```

En el método `load` es dónde añadimos el código que se encarga de crear los objetos que nos interesan y guardarlos en la base de datos. Modificamos el archivo para añadir un par de apuestas:

```
<?php

namespace App\DataFixtures;

use App\Entity\Apuesta;
use Doctrine\Bundle\FixturesBundle\Fixture;
use Doctrine\Persistence\ObjectManager;

class ApuestaFixtures extends Fixture
{
    public function load(ObjectManager $manager)
    {
        $apuesta = new Apuesta();
    }
}
```

```
$apuesta->setTexto('4 8 15 16 23 42');  
$apuesta->setFecha(new \DateTime('now'));  
$manager->persist($apuesta);  
  
$apuesta = new Apuesta();  
$apuesta->setTexto('2 4 14 15 31 10 12');  
  
$manager->persist($apuesta);  
  
$manager->flush();  
    }  
}
```

Para cargar las *fixtures* en la base de datos ejecutamos el siguiente comando:

```
php bin/console doctrine:fixtures:load
```

Ejercicio 3: Autenticación

¡Atención!

Usaremos Fixtures, de modo que es necesario realizar antes el ejercicio de ampliación 3.

La autenticación y el control de acceso es algo que gran parte de las aplicaciones web necesitan, por lo que no era de extrañar que Symfony incluyera un componente destinado a ello. En primer lugar instalamos el componente de seguridad:

```
composer require symfony/security-bundle
```

Usuario

Hay varias formas de proporcionar usuarios a la aplicación. La más común es crear una entidad Usuario, que almacenaremos en la BD. Para ello Symfony proporciona el siguiente comando:

```
php bin/console make:user
```

En este caso ejecutamos el comando y vamos a crear una entidad llamada usuario:

```
The name of the security user class (e.g. User) [User]:
> Usuario

Do you want to store user data in the database (via Doctrine)? (yes/no) [yes]:
> yes

Enter a property name that will be the unique "display" name for the user (e.g.
email, username, uuid) [email]:
> email

Will this app need to hash/check user passwords? Choose No if passwords are not
needed or will be checked/hashed by some other system (e.g. a single sign-on
server).

Does this app need to hash/check user passwords? (yes/no) [yes]:
> yes
```

Si necesitamos añadir más campos a la entidad Usuario, podemos hacerlo con el comando `make:entity` que ya conocemos:

```
php bin/console make:entity
```

Vamos a ejecutar el comando y añadir un campo nombre:

```
Class name of the entity to create or update (e.g. OrangePopsicle):
> Usuario

Your entity already exists! So let's add some new fields!

New property name (press <return> to stop adding fields):
> nombre

Field type (enter ? to see all types) [string]:
```



```
>

Field length [255]:
>

Can this field be null in the database (nullable) (yes/no) [no]:
>
```

Ya tenemos la entidad usuario creada. Ahora debemos crear la migración correspondiente y aplicarla para que se actualice la base de datos:

```
php bin/console make:migration
php bin/console doctrine:migrations:migrate
```

Autenticación

Recordemos que autenticar es comprobar que un usuario es quien dice ser. Symfony proporciona varios métodos: autenticación HTTP, LDAP, mediante certificado... pero utilizaremos la más común: mediante un formulario HTML.

Para ello, Symfony proporciona un asistente que se encarga de crear controladores, rutas, plantillas... de forma interactiva:

```
php bin/console make:auth
```

Ejecutamos el script, respondiendo de la siguiente manera:

```
What style of authentication do you want? [Empty authenticator]:
  [0] Empty authenticator
  [1] Login form authenticator
> 1

The class name of the authenticator to create (e.g. AppCustomAuthenticator):
> LoginFormAuthenticator

Choose a name for the controller class (e.g. SecurityController)
[SecurityController]:
> SecurityController

Do you want to generate a '/logout' URL? (yes/no) [yes]:
> yes
```

El asistente:

- Ha creado un autenticador por fomulario de login (LoginFormAuthenticator).
- Ha creado un controlador que se encargue de los inicios y cierres de sesión (SecurityController). La configuración de las rutas se incluye como anotaciones en el controlador, de forma que el fichero `routes.yaml` no se modifica.
- Ha creado tanto una URL para iniciar sesión (login) como una para cerrarla (logout).

- Ha creado una plantilla `templates/security/login.html.twig` para el formulario de login.

¡Ahora tú!

La plantilla `templates/security/login.html.twig` se basa en la plantilla `base.html.twig` que incluye Symfony de serie. Modificala para que se base en la plantilla `templates/sorteo/estructuraBase.html.twig`.

Tenemos que modificar el autenticador (`src/Security/LoginFormAuthenticator.php`) para indicarle a dónde enviamos al usuario cuando este inicia sesión correctamente. En nuestro caso lo mandaremos a la portada (ruta `index`), de forma que modificamos el método `onAuthenticationSuccess`:

```
public function onAuthenticationSuccess(Request $request, TokenInterface
$token, string $providerKey)
{
    if ($targetPath = $this->getTargetPath($request->getSession(),
$providerKey)) {
        return new RedirectResponse($targetPath);
    }

    // For example : return new RedirectResponse($this->urlGenerator-
>generate('some_route'));
    throw new \Exception('TODO: provide a valid redirect inside '.__FILE__);
}
```

Para que quede de la siguiente manera:

```
public function onAuthenticationSuccess(Request $request, TokenInterface
$token, string $providerKey)
{
    if ($targetPath = $this->getTargetPath($request->getSession(),
$providerKey)) {
        return new RedirectResponse($targetPath);
    }

    return new RedirectResponse($this->urlGenerator->generate('index'));
}
```

De esa manera ya tenemos listo nuestro sistema de autenticación. Tendremos un formulario en `/login` y para cerrar sesión tendremos que redirigir la usuario a `/logout`

Crear usuarios por defecto

Para crear un usuario simplemente hay que crear un objeto de la clase `Usuario`, rellenarlo y persistirlo. Podemos diseñar un formulario para crear usuarios, pero una forma rápida y eficaz de crear usuarios por defecto es mediante `Fixtures`, de modo que ejecutamos el comando:

```
php bin/console make:fixtures
```

Y respondemos lo siguiente:

```
The class name of the fixtures to create (e.g. AppFixtures):  
> UsuarioFixtures
```

Rellenamos el archivo `src/DataFixtures/UsuarioFixtures.php` para crear dos usuarios, uno normal y otro administrador:

```
namespace App\DataFixtures;  
  
use App\Entity\Usuario;  
use Doctrine\Bundle\FixturesBundle\Fixture;  
use Doctrine\Persistence\ObjectManager;  
use Symfony\Component\Security\Core\Encoder\UserPasswordEncoderInterface;  
  
class UsuarioFixtures extends Fixture  
{  
    private $passwordEncoder;  
  
    public function __construct(UserPasswordEncoderInterface $passwordEncoder)  
    {  
        $this->passwordEncoder = $passwordEncoder;  
    }  
  
    public function load(ObjectManager $manager)  
    {  
        // Usuario normal  
        $usuario = new Usuario();  
        $usuario->setNombre("Laura Moreno");  
        $usuario->setEmail("laury28@gmail.com");  
        $usuario->setPassword($this->passwordEncoder->encodePassword(  
            $usuario,  
            'servilleta' // La contraseña  
        ));  
        $manager->persist($usuario);  
  
        // Usuario administrador  
        $usuario = new Usuario();  
        $usuario->setNombre("Manuel Gómez");  
        $usuario->setEmail("manolo30@gmail.com");  
        $usuario->setPassword($this->passwordEncoder->encodePassword(  
            $usuario,  
            'Sevilla12'  
        ));  
        // Le damos el rol de administrador (ROLE_ADMIN).  
        $usuario->setRoles(array("ROLE_ADMIN"));  
        $manager->persist($usuario);  
  
        $manager->flush();  
    }  
}
```

Fíjate en lo siguiente:

- Para fijar la contraseña no la podemos indicar directamente, sino que necesitamos declarar y utilizar el `passwordEncoder`, que es el servicio que se encarga de cifrar las contraseñas según el método configurado (MD5, SHA1...).

•Al administrador se le ha añadido el rol `ROLE_ADMIN`, mientras que al usuario no se le ha añadido nada. Si observamos la entidad `Usuario`, encontramos las siguientes líneas:

```
public function getRoles(): array
{
    $roles = $this->roles;
    // guarantee every user at least has ROLE_USER
    $roles[] = 'ROLE_USER';

    return array_unique($roles);
}
```

•Todas los objetos de la clase `Usuario` devuelven el rol `ROLE_USER`, de forma que no hace falta añadirlo (todos son usuarios registrados). Los nombres de estos roles son personalizables (podemos usar `ROLE_AMIGO`, `ROLE_SUPERADMIN`...) pero la convención es que empiecen por `ROLE_`.

Por último cargamos las fixtures:

```
php bin/console doctrine:fixtures:load
```

Restringiendo el acceso

Para restringir el acceso hay varias maneras:

Restringir por patrón de ruta

Podemos restringir el acceso a las rutas que tengan una estructura similar modificando el archivo `config/packages/security.yaml` y usando expresiones regulares. Por ejemplo vamos a añadir una regla para que las rutas dentro de `/apuesta/` sólo sean accesibles a usuarios registrados y otras para que sólo el administrador pueda añadir, editar o borrar apuestas:

```
access_control:
    - { path: '^/sorteo/apuestas/nueva', roles: ROLE_ADMIN }
    - { path: '^/apuesta/(editar|borrar)/', roles: ROLE_ADMIN }
    - { path: '^/apuesta/', roles: ROLE_USER }
```

¡Atención!

Hay que tener cuidado con el orden de las restricciones. Symfony las comprueba de arriba a abajo y, si una da acceso, no comprueba las demás.

Si intentamos acceder sin registrarnos, Symfony nos enviará al formulario de login automáticamente y, cuando iniciemos sesión, nos dejará ver la página correspondiente.

Para crear una apuesta nueva ocurrirá lo mismo, y si ya estamos registrados como usuarios nos devolverá un error 403 Forbidden.

Restringir en twig

En twig podemos comprobar los roles del usuario y usarlos dentro de condiciones con `is_granted()`. Por ejemplo vamos a incluir un enlace para cerrar sesión en el menú que se mostrará si el usuario está registrado, y un enlace para iniciar sesión en caso contrario:

```
{# templates/sorteo/menu.html.twig #}  
<h2>Menu</h2>  
<ul>  
  <li {% if seccionActual == "inicio" %}class="actual"{% endif %}>  
    <a href="{{ path("index") }}">Inicio</a>  
  </li>  
{# ... #}  
</ul>  
  
{% if is_granted('ROLE_USER') %}  
<p><a href="{{ path("app_logout") }}">Cerrar sesión</a></p>  
{% else %}  
<p><a href="{{ path("app_login") }}">Iniciar sesión</a></p>  
{% endif %}
```

Para saber más

Los nombres de las rutas de login y logout no se encuentran en `routes.yaml`, sino en el controlador `SecurityController`, ya que las rutas están configuradas como anotaciones:

```
// ...  
class SecurityController extends AbstractController  
{  
    /**  
     * @Route("/login", name="app_login")  
     */  
    public function login(AuthenticationUtils $authenticationUtils): Response  
    {  
        // ...  
    }  
  
    /**  
     * @Route("/logout", name="app_logout")  
     */  
    public function logout()  
    {  
        // ...  
    }  
}
```

Ahora tú!

Modifica el enlace de añadir apuesta del menú y los de editar y borrar apuesta que hay en la página de ver apuestas para que sólo se muestren si el usuario tiene el rol de administrador.

Obteniendo el usuario actual

En el controlador

En el controlador podemos acceder al objeto del usuario que está registrado de la siguiente manera:

```
$usuario = $this->getUser();  
echo $usuario->getNombre();
```

En las plantillas

En twig podemos acceder al objeto del usuario registrado en la variable global `app.user`. Por ejemplo vamos a modificar el final de `menu.html.twig`, donde añadimos el enlace para cerrar sesión, para mostrar también información sobre el usuario:

```
{% if is_granted('ROLE_USER') %}  
<h3>Hola {{ app.user.nombre }}</h3>  
<p><small>Sesión iniciada como {{ app.user.email }}</small></p>  
<p><a href="{{ path("app_logout") }}">Cerrar sesión</a></p>  
{% else %}  
<p><a href="{{ path("app_login") }}">Iniciar sesión</a></p>  
{% endif %}
```